Objektno orijentirano programiranje
Vježba 5

1. Napišite klasu Volume koja predstavlja glasnoću zvuka. Glasnoća je cijeli broj od 0 do 100 i to je invarijanta klase.
   - Konstruktor Volume(int l) vrijednost izvan raspona svodi na najbližu granicu (-20 postaje 0, 130 postaje 100).
   - Default konstruktor postavlja glasnoću na 50 delegiranjem na prvi konstruktor.
   - Napišite pomoćnu static funkciju koja ograničava vrijednost na 0–100.
   - Metode up(int step = 10) i down(int step = 10) mijenjaju glasnoću, ali nikad izvan raspona.
   - mute() postavlja glasnoću na 0, a unmute() vraća glasnoću na vrijednost prije mute().
   - Napišite privatnu metodu bool valid() const i provjerite invarijantu s assert na kraju svake metode koja mijenja glasnoću.

   Program za testiranje je u datoteci volume_main.cpp (očekivani ispis je na dnu datoteke).

2. Klasa Clip predstavlja isječak pjesme od sekunde start do sekunde end. Zadana je u datoteci clip.cpp, zajedno s main:

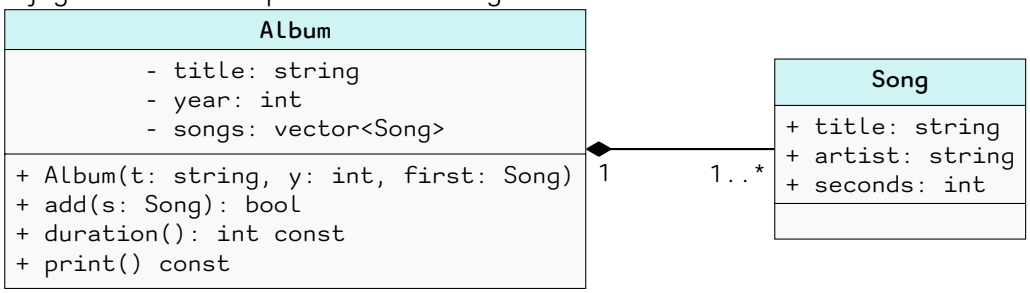
```
class Clip {
    int length;
    int start;
    int end;
    std::string name;
    const int max_length;
public:
    Clip(const std::string& name, int s, int e)
        : start(s), end(e), length(end - start)
    {
        name = name;
        max_length = 600;
    }
    void print() const {
        std::cout << name << ": " << length << " s\n";
    }
};

int main() {
    Clip c("Concrete (Shame)", 10, 190);
    c.print();
}
```

   Ispravite klasu tako da program ispiše

```
Concrete (Shame): 180 s
```

   Za svaku grešku objasnite zašto nastaje.
   Hint 1: Dvije greške javlja kompajler. Pogledajte na predavanju koji članovi moraju dobiti vrijednost pri nastanku i što znači this->.
   Hint 2: Treća greška nije greška pri kompajliranju: program se prevede, ali ispiše krivo trajanje. g++ uz -Wall na nju upozori, a Visual Studio ne upozori ni na razini Level4 (/W4). Usporedite redoslijed članova u klasi s redoslijedom u inicijalizacijskoj listi.
   Postavke kompajlera za warninge:
   g++: g++ -Wall -Wextra clip.cpp -o clip
   Visual Studio: Project → Properties (Configuration: All Configurations) → C/C++ → General → Warning Level → Level4 (/W4)
   VS Code: u datoteci .vscode/tasks.json u popis "args" dodati "-Wall", "-Wextra"

3. Prema dijagramu klasa napišite klase Song i Album.



   - Multiplicitet `1..*` je invarijanta: album nikad nije prazan. Zato konstruktor prima prvu pjesmu.
   - `add` ne dodaje pjesmu ako album već ima pjesmu istog naslova i tada vraća `false`.
   - `duration()` vraća ukupno trajanje u sekundama, a `print()` ispisuje album u obliku kao ispod.

   Program za testiranje je u datoteci `album_main.cpp` (očekivani ispis je na dnu datoteke).
   Kakva je veza između albuma i pjesama: kompozicija ili asocijacija? Kako se to vidi u kodu?

4. Igra se sastoji od jednog igrača i jednog levela. Level ima barem jednog neprijatelja, a može imati i oružja. Igrač u svakom trenutku zna u kojem je levelu i može nositi najviše jedno oružje. Kad igrač podigne oružje, ono se uklanja iz levela i pripada igraču.
   - Nacrtajte dijagram klasa (`Game`, `Player`, `Level`, `Enemy`, `Weapon`) s vezama i multiplicitetima. Za svaku vezu odlučite je li kompozicija ili asocijacija. Dijagram se može nacrtati na papiru ili napisati kao tekst u PlantUML-u.
   - Dopunite kostur klasa u mapi `igra`: za svaku klasu postoje `.h` i `.cpp` datoteka s potrebnim `#include`-ima. Napišite data membere, konstruktore i metode koje su već deklarirane u kosturu. Iz koda mora biti vidljivo koja je veza kompozicija, a koja asocijacija.
   - Oružje igrača (najviše jedno) nacrtajte u dijagramu, ali ga u kodu ne pišite: data member uvijek postoji, pa ne može izraziti "nula ili jedno oružje".
   - Sve datoteke prevode se zajedno: `g++ -Wall -Wextra *.cpp -o igra`. U Visual Studiju sve datoteke treba dodati u isti projekt.

   Program za testiranje je u datoteci `main.cpp` (očekivani ispis je na dnu datoteke).

5. Napišite klasu `Ticket` (ulaznica za koncert) s nazivom koncerta i jedinstvenim brojem.
   - Broj se dodjeljuje automatski u konstruktoru: prva ulaznica ima broj 1, sljedeća 2, itd. Koristite statički član (definicija izvan klase).
   - Statička metoda `issued()` vraća broj dosad izdanih ulaznica.

   Program za testiranje je u datoteci `ticket_main.cpp` (očekivani ispis je na dnu datoteke).

6. Napišite klasu `Coin` (novčić u igri) s vrijednošću i statičkim brojačem novčića koji trenutno postoje. Novčić postoji dok ga igrač ne pokupi.
   - Konstruktor povećava brojač, a destruktor ga smanjuje.
   - Statička metoda `remaining()` vraća broj novčića koji trenutno postoje.

   Za razliku od `Ticket`, koji broji koliko je ulaznica izdano, ovdje se broje novčići koji još postoje. Program za testiranje je u datoteci `coin_main.cpp` (očekivani ispis je na dnu datoteke).